Rockchip DRM Direct Show Developer Guide

ID: RK-YH-YF-454

Release Version: V1.1.0

Release Date: 2026-01-10

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

This article mainly introduces the basic characteristics, workflow, and common problem analysis of the Rockchip platform processor's VOP based on the DRM display framework and related display interfaces. The purpose is to enable relevant engineers to have a better understanding of the DRM display driver framework and hardware interfaces, and to quickly locate and solve problems through the analysis of common problems.

Product Version

Chip Name	Kernel Version
RK3528/RK3566/RK3562/RK3568/RK3588	Linux kernel 5.10 and above
RK3538/RK3576	Linux kernel 6.1 and above

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Hardware development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Sandy Huang	2022-03-01	Initial version
V1.1.0	Sandy Huang	2026-01-10	Add more platform and API support

Table of Contents

Rockchip DRM Direct Show Developer Guide

1. Direct Show API and Usage Instructions
    - 1.1 API Description
        - 1.1.1 Obtain DRM Device
        - 1.1.2 Allocate Memory
        - 1.1.3 get crtc
        - 1.1.4 get plane
        - 1.1.5 commit display
        - 1.1.6 close plane
        - 1.1.7 free buffer
        - 1.1.8 cache invalid
        - 1.1.9 cache flush
    - 1.2 Usage Process
2. Automated Test Case Description
3. Kernel Logo Display Description

1. Direct Show API and Usage Instructions

1.1 API Description

1.1.1 Obtain DRM Device

```
struct drm_device *rockchip_drm_get_dev(void)
```

This API is used to obtain the DRM device handle. The acquired DRM device is a prerequisite for subsequent processes including memory allocation and release, CRTC and plane acquisition, and display commit operations.

1.1.2 Allocate Memory

```
int rockchip_drm_direct_show_alloc_buffer(struct drm_device *drm,
                                          struct rockchip_drm_direct_show_buffer *buffer)
```

- input buffer

Variable	Description
width	Width of the buffer, in pixels
height	Height of the buffer, in pixels
pixel_format	Format of the buffer; refer to `include/uapi/drm/drm_fourcc.h` for specific definitions
flag	Used to specify the buffer type. Default value `0` indicates discrete non-contiguous memory; `ROCKCHIP_BO_CONTIG` indicates contiguous memory

- output buffer

Variable	Description
bpp	Number of bits per pixel obtained based on buffer->pixel_format
pitch[3]	Virtual width of the current buffer obtained based on `buffer->width`, `buffer->pixel_format` and certain alignment rules
vir_addr[3]	Kernel-space virtual address of the currently allocated buffer
phy_addr[3]	Physical address of the currently allocated buffer
rk_gem_obj	Abstraction used by the Rockchip DRM GEM driver to manage the current buffer
fb	Abstraction of the current buffer in DRM plane display
dmabuf_fd	dmabuf_fd exported from the current buffer, which may be used by other modules

Notes on `buffer->pitch[3]`, `buffer->vir_addr[3]` and `buffer->phy_addr[3]`:

Take `buffer->pitch[3]` as an example. For RGB formats, `buffer->pitch[0]` is used by default. For the NV12 format which has 2 planes, `buffer->pitch[0]` and `buffer->pitch[1]` are used. For formats with 3 planes, `buffer->pitch[0]`, `buffer->pitch[1]` and `buffer->pitch[2]` are used.

1.1.3 get crtc

```
struct drm_crtc *rockchip_drm_direct_show_get_crtc(struct drm_device *drm)
```

Used to obtain the currently active CRTC.

1.1.4 get plane

```
struct drm_plane *rockchip_drm_direct_show_get_plane(struct drm_device *drm, char *name)
```

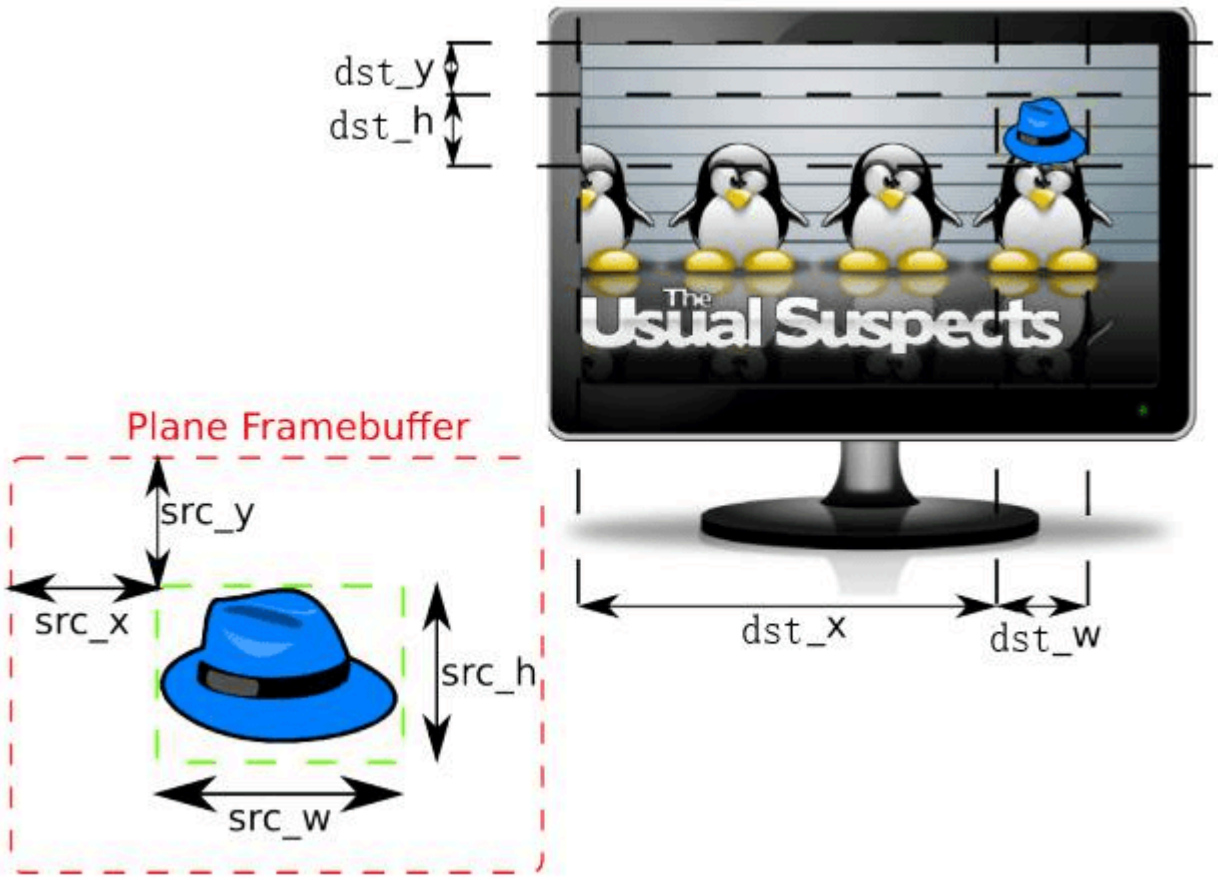
Used to obtain the corresponding plane based on the name information. The acquired plane will be used for display purposes. For example, to obtain Esmart0-win0:

```
plane = rockchip_drm_direct_show_get_plane(drm_dev, "Esmart0-win0")
```

1.1.5 commit display

```
int rockchip_drm_direct_show_commit(struct drm_device *drm,
                                    struct rockchip_drm_direct_show_commit_info
*commit_info)
```

commit_info Variable	Description
crtc	Specifies the CRTC to submit to
plane	Specifies the plane to submit to
buffer	Buffer submitted for display
top_zpos	false: use default configuration; true: set to the topmost layer
src_x/src_y/src_w/src_h	Offset and size of the display area within the buffer; refer to the figure below for details
dst_x/dst_y/dst_w/dst_h	Position and size of the current layer displayed on the screen; refer to the figure below for details



1.1.6 close plane

```
int rockchip_drm_direct_show_disable_plane(struct drm_device *drm, struct drm_plane *plane)
```

If the display of the current layer content is no longer required, this interface can be used to disable the layer display.

1.1.7 free buffer

```
void rockchip_drm_direct_show_free_buffer(struct drm_device *drm,
                                          struct rockchip_drm_direct_show_buffer *buffer)
```

After the layer is disabled, if the buffer is no longer needed, this interface can be used to release and reclaim the memory.

1.1.8 cache invalid

In certain application scenarios, the CPU is required to operate on image data. To improve efficiency, a cache-enabled buffer is allocated. In this case, **cache coherence** must be ensured. Before the CPU operates on the buffer, invalidate the corresponding cache lines of the buffer to ensure that the CPU reads the latest data from DDR.

```
int rockchip_drm_direct_show_buf_begin_cpu_access(struct rockchip_drm_direct_show_buffer
*buffer)
```

Method for Allocating a Cache-enabled Buffer:

```
--- a/drivers/gpu/drm/rockchip/rockchip_drm_self_test.c
+++ b/drivers/gpu/drm/rockchip/rockchip_drm_self_test.c
@@ -93,7 +93,7 @@ static int rockchip_drm_self_test_alloc_buffer(struct
rockchip_drm_self_test *se
                buffer->width = BUFFER_WIDTH;
                buffer->height = BUFFER_HEIGHT;
                buffer->pixel_format = BUFFER_FORMAT;
-               buffer->flag = ROCKCHIP_BO_CONTIG;
+               buffer->flag = ROCKCHIP_BO_CONTIG | ROCKCHIP_BO_CACHABLE;
                ret = rockchip_drm_direct_show_alloc_buffer(self_test->dev, buffer);
```

1.1.9 cache flush

After the CPU finishes operating on the buffer, flush the data in the corresponding cache lines of the buffer to DDR to ensure that the DMA can retrieve the latest data.

```
int rockchip_drm_direct_show_buf_end_cpu_access(struct rockchip_drm_direct_show_buffer
*buffer)
```

1.2 Usage Process

Rockchip DRM Direct Show is disabled by default. To enable this feature, you can enable it in `rockchip_defconfig`:

```
diff --git a/arch/arm64/configs/rockchip_defconfig b/arch/arm64/configs/rockchip_defconfig
index b1c92060c6f9..2e3091f8631f 100644
--- a/arch/arm64/configs/rockchip_defconfig
+++ b/arch/arm64/configs/rockchip_defconfig
@@ -588,6 +588,8 @@ CONFIG_DRM_IGNORE_IOTCL_PERMIT=y
 CONFIG_DRM_DP_AUX_CHARDEV=y
 CONFIG_DRM_LOAD_EDID_FIRMWARE=y
 CONFIG_DRM_ROCKCHIP=y
+CONFIG_ROCKCHIP_DRM_DIRECT_SHOW=y
```

The usage process can basically be carried out in the following 7 steps:

1. Obtain the DRM device;
2. Allocate a buffer of the corresponding format and size as required;
3. Draw the required image into the allocated buffer;
4. Obtain the CRTC and plane;
5. Commit the display;
6. Disable the layer if the display is no longer needed;
7. Release and reclaim the memory.

2. Automated Test Case Description

An automated test program for Rockchip DRM has been implemented based on Rockchip DRM Direct Show. It can be used as a reference for developers, as well as an automated test program for the Rockchip DRM driver. This feature is disabled by default and can be enabled in `rockchip_defconfig`.

```
diff --git a/arch/arm64/configs/rockchip_defconfig b/arch/arm64/configs/rockchip_defconfig
index b1c92060c6f9..2e3091f8631f 100644
--- a/arch/arm64/configs/rockchip_defconfig
+++ b/arch/arm64/configs/rockchip_defconfig
@@ -588,6 +588,8 @@ CONFIG_DRM_IGNORE_IOTCL_PERMIT=y
 CONFIG_DRM_DP_AUX_CHARDEV=y
 CONFIG_DRM_LOAD_EDID_FIRMWARE=y
 CONFIG_DRM_ROCKCHIP=y
 CONFIG_ROCKCHIP_DRM_DIRECT_SHOW=y
+CONFIG_ROCKCHIP_DRM_SELF_TEST=y
```

3. Kernel Logo Display Description

For products that do not use Rockchip's official U-Boot, or cannot enable U-Boot logo display due to certain reasons, but still require kernel logo display, this function can be implemented by modifying the Direct Show automated test program.

```
diff --git a/drivers/gpu/drm/rockchip/rockchip_drm_self_test.c
b/drivers/gpu/drm/rockchip/rockchip_drm_self_test.c
index 74c395441e96..355632125914 100644
--- a/drivers/gpu/drm/rockchip/rockchip_drm_self_test.c
+++ b/drivers/gpu/drm/rockchip/rockchip_drm_self_test.c
@@ -20,7 +20,7 @@
 #include "kernel_logo_img.h"

-#define USE_BUFFER_NUM 2
+#define USE_BUFFER_NUM 1
 #define BUFFER_WIDTH   652
 #define BUFFER_HEIGHT  268
 #define BUFFER_FORMAT  DRM_FORMAT_RGB565 /*
DRM_FORMAT_RGB565/DRM_FORMAT_XRGB8888/DRM_FORMAT_NV12 */
@@ -159,7 +159,7 @@ static void rockchip_drm_self_test_commit(struct work_struct *work)
                goto free_buffer;
        }

-#if 1  /* for self test pattern */
+#if 0  /* for self test pattern */
        /* commit to display */
        do {
                u32 i = 0;
```

`kernel_logo_img.h` is a file that stores the converted data from BMP files using tools such as bmp2hex. Modify the macro definitions of **BUFFER_WIDTH**, **BUFFER_HEIGHT** and **BUFFER_FORMAT** according to the actual image used.

If you cannot find a suitable bmp2hex tool, you may search for **bmp2hex** to obtain it from the following link on Rockchip Redmine: https://redmine.rock-chips.com/documents/97.